SYNTHESIZABLE VHDL • T65 CORE • HARDWARE–VERIFIED

A 6502 Home Computer Rebuilt in Logic

One machine, two implementations — an FPGA system and a software emulator sharing a single memory map and ROM set

Rudolf Stepan
Independent hardware & systems research
ORCID: 0009-0004-2842-2579
stepan.science • intracom.at/projects/6502_fpga_computer

Abstract

I have been building an 8-bit home computer around the MOS 6502, and this paper reports where it stands. It is not a clone of any particular machine. The aim is a usable and extensible platform that still runs the existing body of 6502 software. There are two implementations, kept in step: an emulator written in C99, and synthesizable VHDL that runs on real FPGA hardware. They agree on one memory map and use the same ROM images, so EhBASIC and ordinary 6502 programs behave the same in simulation and on the board. On the FPGA the peripheral chips are written in RTL rather than dropped in as opaque IP: a 6502 CPU core (T65), a C64-style video controller with text and bitmap modes, a SID-compatible (MOS 6581) sound chip, 6522/6551 serial and timer I/O, a CIA-1 timer, and a disk controller that reads FAT32 and D64 images in hardware. VIC-II colour and raster registers at their C64 addresses, together with a short build tool, let it play a wide range of original `.sid` tunes from their own player code, interrupt- and raster-timed players included. The machine boots from SD card and draws FPU-accelerated Mandelbrot images over HDMI on a Sipeed Tang Primer 20K; the same core also runs on a Xilinx Spartan-6 board.

Project paper • June 27, 2026• Source: github.com/rudolfstepan/6502-sbc-fpga

1 Introduction

The MOS 6502 is still one of the best-understood microprocessors around, and there is a large amount of software for it that people continue to write and maintain. New 6502 projects mostly land in one of two groups. Some recreate a specific old computer as faithfully as possible: a Commodore 64, an Apple II, a BBC Micro. Others are bare single-board machines with a CPU, some RAM and a serial port, and not much else.

I wanted something in between. The point of this project is not to copy a Commodore, but to design a 6502 home computer of my own that is comfortable to use today and still compatible with classic 6502 software where that makes sense. I define the machine once, as a memory map plus a set of peripheral register contracts, and then build it twice: as a software emulator and as synthesizable logic for FPGAs. Since both honour the same addresses and registers, the same ROM images and the same programs run on either without changes.

The rest of the paper walks through the architecture, the individual subsystems, how each part is tested, and how far the hardware bring-up has come.

2 Design Goals

A few decisions shaped everything that followed.

One machine, two implementations.
The emulator and the FPGA design are not two similar-looking projects. They are the same machine built two ways, and the memory map is the contract that keeps them honest.

Run unmodified 6502 software.
EhBASIC and standalone ROM applications have to boot as-is. Whatever I write and debug in the emulator should run on the board, and the other way round.

Write the chips in RTL.
Video, sound, timers, serial and disk are re-implemented as a board-independent VHDL library with the original registers at the original addresses, instead of being wrapped around opaque vendor IP.

Be modern where it actually helps.
Some conveniences the 6502 era did without are worth having: HDMI output, an SD card for storage, a hardware multiplier. Each is reached through a plain memory-mapped interface the CPU can drive.

Test before trusting.
Every block has to pass in simulation before it goes near the FPGA.

3 System Overview

Figure 1 shows the FPGA system. A 6502-compatible CPU core sits on a 16-bit address bus, and a combinational address decoder routes each access to main RAM, to the banked video framebuffer, or to one of the memory-mapped peripherals. Before any of that runs, a boot subsystem copies the system ROM from the SD card into writable shadow RAM and only then releases the CPU from reset.

The same specification also drives the reference emulator, a C99 program with an SDL2 backend and an interactive monitor. The FPGA repository pulls in the emulator's ROM and kernel sources directly, so the two tracks cannot drift apart without someone noticing at build time. Figure 2 shows the FPGA build sitting at the EhBASIC prompt.

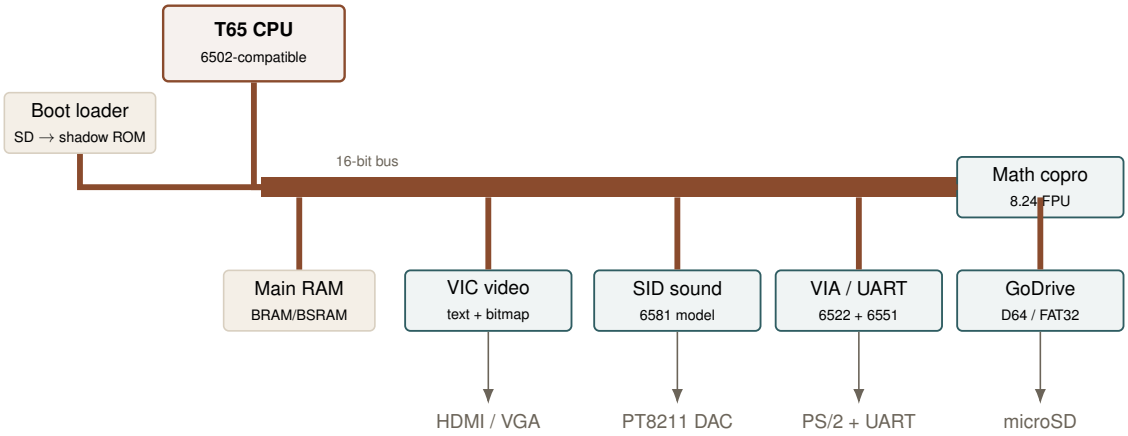


Figure 1: *Block diagram of the FPGA system. The board-independent RTL core (CPU, bus, RAM, and the memory-mapped peripheral library) is shared across both FPGA targets; only thin board glue differs.*

Figure 2: *Enhanced BASIC booting on the HDMI output of the Tang Primer 20K. The banner is drawn by the kernel's self-test before control passes to the BASIC interpreter; the same ROM image runs unchanged in the emulator.*

4 Processor Core

The CPU is the open-source **T65** soft core, a 6502-compatible processor written in VHDL. A small project-local adapter sits between the core and the system: it narrows the core's wide internal address bus to 16 bits, exports the write-enable and write-data signals, and feeds read data from memory and peripherals back in. A `cpu_enable` toggle lets the processor step on every other system clock, which gives an effective CPU rate of up to **27 MHz** and still leaves a settled bus phase for the synchronous FPGA memories. Interrupts work as on the real part: the reset vector comes from `$FFFC`, and maskable interrupts vector through `$FFFE`/`$FFFF`.

That half-rate stepping is the trick that makes the rest easy. The 6502 bus expects asynchronous-style reads, but block RAM on an FPGA is synchronous. Registering read data during the settled half of each cycle reconciles the two and keeps the simulation timing deterministic.

5 Memory Map

If anything is the heart of the project, it is the memory map: the one contract both implementations honour. Table 1 lists the configuration currently running on the Tang Primer 20K.

Range	Size	Device
$0000–$3FFF	16 KB	BRAM main RAM — zero page, stack, EhBASIC
$4000–$5FFF	8 KB	BSRAM main RAM (optional DDR3 backend)
$6000–$7FFF	8 KB	Window into the banked VIC framebuffer
$8000–$87FF	2 KB	VIC text & colour RAM
$8800–$880F	16 B	VIA 6522
$8810–$8813	4 B	UART 6551
$8820–$8823	4 B	Keyboard (PS/2 controller; USB-HID prepared)
$8824	—	D64 GoDrive registers
$88B0–$88BF	16 B	Math coprocessor (8.24 FPU)
$9000–$900F	16 B	VIC control registers
$D000–$D03F	64 B	VIC-II registers (border/bg + $D011/$D012 raster)
$A000–$CFFF	12 KB	Shadow-ROM application window
$D400–$D41C	29 B	SID-compatible audio registers (incl. OSC3/ENV3)
$DC00–$DC0F	16 B	CIA-1 — Timer A + interrupt control
$F000–$FFFF	4 KB	Shadow-ROM kernel & vectors

Table 1: *Active Tang Primer 20K memory map. The C64-compatible VIC-II ($D020/$D021, $D011/$D012), SID ($D400) and CIA-1 ($DC00) blocks sit at their original addresses, so classic `POKE`s and SID players work unchanged. The physical shadow-ROM image is 16 KB: offsets `$0000–$2FFF` map to `$A000–$CFFF`, and `$3000–$3FFF` map to `$F000–$FFFF`.*

6 Video Subsystem (VIC)

The video controller re-implements a C64-style display in RTL, with its own horizontal and vertical sync generation, character-ROM lookup, and a 16-colour Pepto-style palette.

6.1 Display modes

The controller supports a text mode and four bitmap modes:

- **40×25 text** — each cell scaled to 16×16 screen pixels, with per-cell foreground/background colour from the colour RAM at `$8400` and an 8×8 glyph from a 256-entry character ROM.
- **320×240, 16 colours (4 bpp)** — a full-screen bitmap held in a dedicated on-chip framebuffer, displayed 1:1 and centred, with the surrounding border filled in the `$D020` colour.
- **320×200 1-bpp** — legacy bitmap, 2× scaled to 640×400, 16 colours per 8×8 cell.
- **160×100 RGB332** — 256 colours, one byte per pixel, 4× scaled.
- **180×120 RGB222** — 64 colours packed four pixels into three bytes, 3× scaled.

The framebuffer (`fb_ram`, 38 400 bytes for 320×240 at 4 bpp) is reached through an 8 KB CPU window at `$6000–$7FFF`; in the 16-colour mode three bank bits in `$9000[7:5]` slide that window across the buffer, while the `$9000` MODE register selects the active display format. Figure 3 shows two of the bitmap modes running on the board, and Figure 5 the text mode alongside the 1-bpp bitmap.

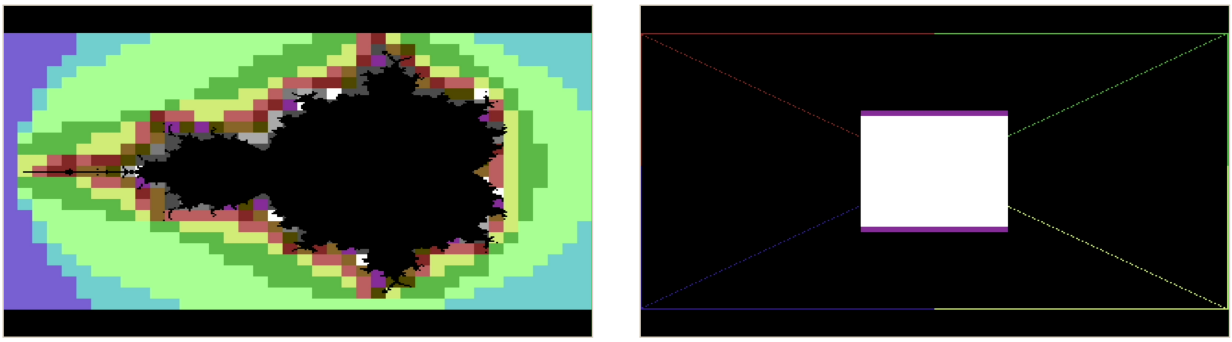


Figure 3: *Bitmap graphics from the running board. Left: an FPU-accelerated 256-colour Mandelbrot set in the RGB332 mode. Right: direct-from-BASIC pixel plotting. Both are photographed from the HDMI output, not rendered.*

6.2 Bus stealing — the shared bus

VIC and CPU share a single-port video memory. During each horizontal blanking interval the VIC prefetches the next display line and holds the CPU through the `RDY` pin. The measured overhead is small — **4.8 %** in text mode, up to 9.4 % in the heaviest bitmap mode — and CPU writes to video RAM are never lost: a pending write is buffered and committed on the first free clock.

6.3 C64-compatible VIC-II registers

For software compatibility the controller also exposes a VIC-II register block at the original C64 addresses, `$D000–$D03F`. The border (`$D020`) and global background (`$D021`) colours respond to the classic `POKE 53280,c` / `POKE 53281,c`, and the rest of the block is a read/write register file. Two registers are *live* on read: `$D012` gives back the current raster line (its low eight bits) and `$D011` bit 7 carries raster bit 8. That small detail earns its place. A raster-synchronised player, or any old timing loop that busy-waits on `$D012`, would otherwise compare against a fixed value and never move on. With the scan counter actually running, the wait completes. Section 7 comes back to this for SID playback. Figure 4 shows the border and background colours set from BASIC with `POKE`.

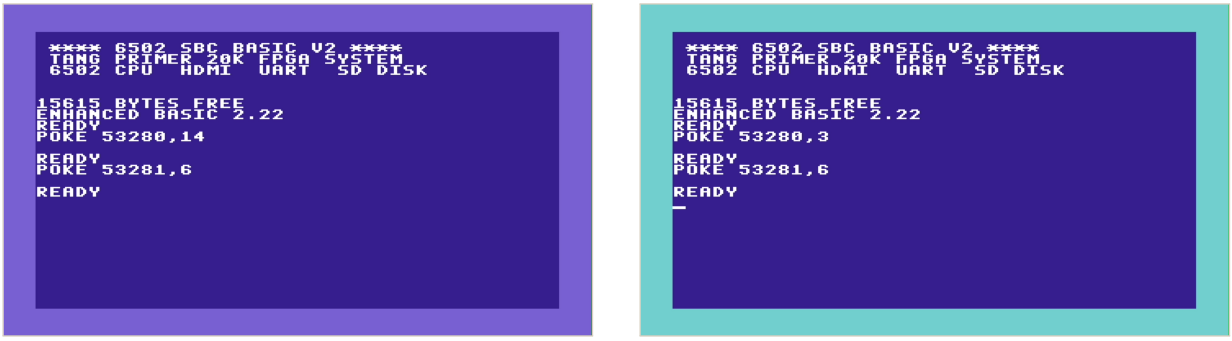


Figure 4: *The C64-compatible colour registers driven from BASIC. `POKE 53280,c` sets the border (`$D020`) and `POKE 53281,c` the background (`$D021`); the two captures use different border colours over the same blue background.*

6.4 Output

On the Tang Primer 20K the controller drives a DVI-style **HDMI** output at **720×480p** (exact CEA-861 480p timing, 640-wide content pillar-boxed) through a TMDS serializer, with a 27 MHz

pixel clock and a 135 MHz TMDS clock. An AVI InfoFrame is emitted alongside the video so that USB HDMI capture devices — which, unlike most monitors, reject a bare DVI signal — lock onto the format. The Spartan-6 reference board instead produces VGA text output with a boot status screen.

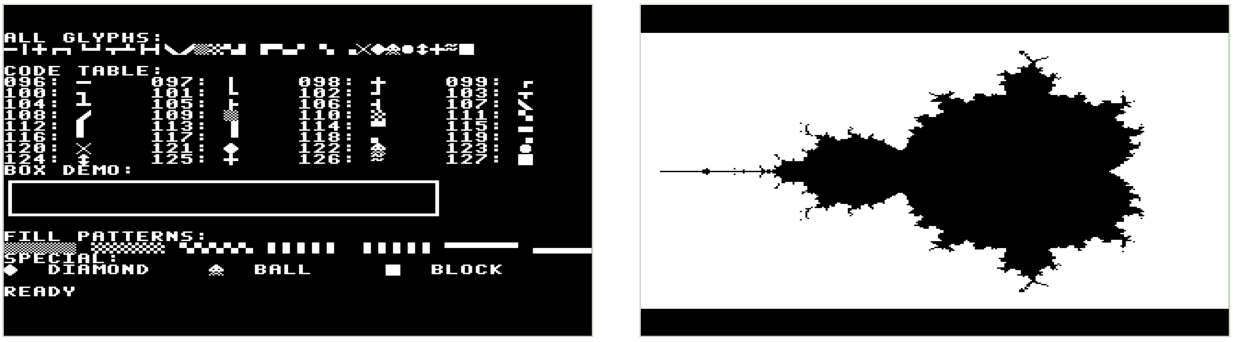


Figure 5: *Left: PETSCII block and line graphics in the 40×25 text mode. Right: a monochrome Mandelbrot in the 320×200 1-bpp bitmap mode.*

7 Audio Subsystem (SID-compatible)

The sound chip is a fairly complete model of the **MOS 6581 (SID)**. It provides three independent voices with combined waveforms, ADSR envelopes built on the reSID rate-counter periods, and a two-pole state-variable multimode filter (low/band/high pass) with a deliberately "dark 6581" cutoff curve. Each oscillator is built on a 24-bit phase accumulator; the register file occupies `$D400–$D41C` in keeping with the original layout (the read-only OSC3/ENV3 registers at `$D41B/$D41C` are included). Audio leaves the FPGA through a PT8211 I^{2}S DAC at roughly 47 kHz.

The hardest thing I ask of it is to play **original `.sid` files together with their 6502 player code**, directly on the hardware. The same routines that would drive a real SID drive this RTL model instead, and the music comes out recognisable. The rest of this section is about how an arbitrary tune gets wrapped to run on the bare machine.

7.1 Wrapping a tune

The tool `build_native_sid_rom.py` turns a PSID/RSID file into a standalone ROM. It does *not* perform a lossy 50 Hz register dump; instead it embeds the tune's original 6502 payload, places it at its native load address, calls the tune's `init` routine once, and then drives the tune's `play` routine. Every frequency, pulse-width, control, ADSR and filter write therefore reaches the hardware exactly as the original player produces it.

Two load layouts are chosen automatically from the tune's load address. A tune that loads into linear RAM (`$0200–$5FFF`) is embedded in a wrapper at `$A000` that copies the payload down to its load address. A tune that loads at `$A000–$CFFF` is handled by a *RAM-under-BASIC* "page" layout: the EhBASIC shadow ROM is made CPU-writable in that window so the tune sits directly at its native address with no copy, and a small player lives at `$F000`.

The `play` address selects one of two drive paths:

Polled (`play` $\neq 0$).
`init` runs once, then `play` is called every ≈20 ms off the millisecond counter at `$883A`. This needs no interrupt and never depends on the CIA or VIC. A free-running CIA Timer A is also started so tunes that *read* `$DC04` see a live timer.

Interrupt-driven (`play` = 0).
The tune installs its own CIA timer and `$0314` handler in `init`; the wrapper supplies only a default vector and a `$FFFE` → `JMP ($0314)` bridge. This path uses the CIA-1 Timer A described in Section 8.

After `init` the wrapper always executes `SEI`. Some RSID `init` routines end with `CLI`, expecting to run under their own interrupt; without re-masking, a stray IRQ would vector through the ROM `$FFFE` bridge into an uninitialised `$0314` and crash. For an ordinary tune whose `init` does not touch the interrupt flag, the `SEI` is a harmless no-op.

7.2 Case study: a raster-timed player

A handful of RSID tunes do not fit either drive path cleanly. They declare `play` = `$0000`, yet they are not silent: their real per-frame update is buried inside a **VIC raster-interrupt** handler. The tune *Arkanoid* is a good example. Disassembling its `init` (`$4000`) shows it switch the C64 memory banks via `$01`, write its own handler vector straight into `$FFFE/$FFFF`, enable a raster interrupt through `$D01A`, and `CLI`. Its handler acknowledges the VIC, programs the next raster compare line, and calls the actual SID update at `$4141`.

None of that survives on the bare machine: the FPGA generates no VIC raster interrupts, `$FFFE/$FFFF` is read-only kernel ROM, and there is no `$01` banking. The fix sidesteps the interrupt entirely. A small per-tune *override table* in the build tool points `play` at the inner update routine (`$4141`) discovered by disassembly, so the *polled* path drives it directly at 50 Hz. Because that tune's `init` performs a `CLI`, the `SEI`-after-`init` guard above is what keeps it from crashing. With both pieces in place the tune plays from its own unmodified player code, with no interrupt at all.

7.3 Building a collection

A companion tool, `build_all_sid_roms.py`, wraps every suitable tune in a folder, emitting a ROM and an upload script for each. By default it first runs each tune through a bare-metal 6502 SID emulator and *skips* tunes that never set the master volume or gate a voice — they would be silent on this hardware. Hand-made demonstration ROMs are protected by a `CURATED` list and are never overwritten by the bulk build.

```
# wrap one tune
python tools/build_native_sid_rom.py path/to/tune.sid sw/<name>.s

# or wrap a whole collection at once (skips silent tunes,
# keeps curated ROMs, links page-mode tunes automatically)
python tools/build_all_sid_roms.py
```

8 Timers, Serial, and Input

Three classic-style chips round out the I/O. The **VIA 6522** (`$8800`) provides timers, general-purpose I/O ports, and interrupt flags; its Timer 1 supplies the kernel's periodic tick, and Port B bit 0 drives the board LED after boot. The **UART 6551** (`$8810`) implements full transmit and receive logic with status registers and a receive interrupt; on the host side it appears as a CH340 serial port at 115200 8N1.

A **CIA-1** (`$DC00–$DC0F`) adds a MOS-6526 subset for C64 compatibility: a Timer A down-counter with latch (`$DC04/$DC05`), a control register (`$DC0E`) for start, one-shot and force-load, and an interrupt control register (`$DC0D`) whose mask is set or cleared by the high bit of the written

value and whose status is cleared on read. This is what lets interrupt-driven SID players raise their player from a Timer-A interrupt exactly as on a real C64.

Keyboard input takes a small shortcut. A **PS/2 keyboard** on a PMOD connector has its keystrokes fed straight into the UART receive path, so EhBASIC and the rest of the firmware read typed characters as ordinary serial input and need no keyboard driver of their own. A generic switches the layout between **German and US**: Y/Z swap, the German shifted number row, AltGr for `@ { [ ] } \ ~ |`, the 102nd <>| key, and umlauts emitted as Latin-1 code points and drawn from a 256-glyph character ROM. I tried a USB-HID bit-bang host first, but it never enumerated reliably, so PS/2 won; the USB-HID register window is *prepared* in the map but is not the active input.

All maskable interrupt sources are OR-combined into the single 6502 `IRQ` line, $\overline{\texttt{irq}} = \neg(\text{via} \vee \text{uart} \vee \text{kbd} \vee \text{disk} \vee \text{cia-1})$, and the CPU vectors through `$FFFE/$FFFF` as usual.

9 Mass Storage: D64 GoDrive

Programs are loaded through a virtual disk drive I call **GoDrive**. A second SD card, formatted FAT32, holds `.d64` disk images. What is unusual is where the parsing happens. The FPGA reads both the FAT32 file system *and* the D64 image format **in logic**, and hands the 6502 nothing but plain 256-byte sectors at `$8824`. No file-system code runs on the CPU at all.

To whoever is typing, it behaves like an old disk drive. The familiar BASIC commands work directly:

```
EhBASIC ready.
LOAD "$"
 0  "TESTDISK     " 1A 2A
   4  "HELLO"          PRG
   8  "MANDELBROT"     PRG
  12  "SOUNDDEMO"      PRG
READY.
LOAD "HELLO"
LOADING... $0801-$08C9
CALL 2049
HELLO FROM A REAL 6502!
READY.
```

The drive autodetects MBR-partitioned cards and "superfloppy" layouts, offers an interactive arrow-key selection menu (`LOAD "!"`), and has been verified on a real 32 GB Windows-formatted SD card. The D64 container is borrowed purely for convenience: the programs it holds are this system's own code and are *not* C64-compatible.

10 Math Coprocessor

Arithmetic-heavy BASIC programs get help from a memory-mapped **FPU** at `$88B0`, which does signed 32×32 fixed-point multiplication in **8.24 format**. That is what makes the Mandelbrot renderings bearable from BASIC: the inner-loop multiplies go to hardware rather than being built up from 8-bit 6502 multiply routines. Figure 6 shows one such render driven from hand-written assembly.

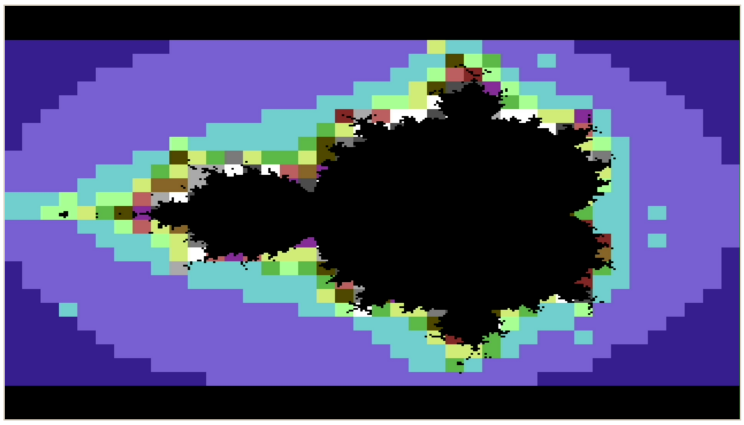
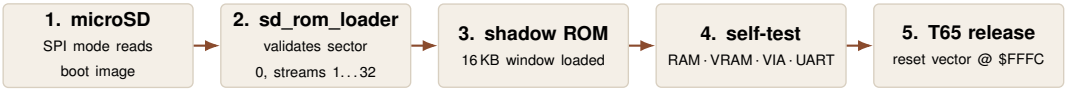


Figure 6: *A Mandelbrot set computed in hand-written 6502 assembly using fixed-point arithmetic, with the inner multiplications handed to the memory-mapped coprocessor.*

11 Boot and Clocking

A single 27 MHz oscillator feeds a 270 MHz PLL, and the TMDS, system and pixel clocks are divided down from there. At power-on the board works through a short sequence before the CPU ever runs:



The boot image survives a reset. While I am working on firmware, a built-in UART monitor can also write straight into the shadow-ROM RAM over the serial link, so a fresh ROM can be uploaded and run without touching the SD card. That copy is volatile: the next reset brings back the SD image.

12 Verification

Every block runs on its own in simulation under **GHDL** before it goes near the FPGA; Table 2 summarises what the testbenches currently cover. The emulator pulls its weight here too. It runs the **Klaus Dormann** 6502 functional test suite as part of its normal build, so the instruction semantics both tracks rely on are checked against an outside reference rather than against my own assumptions.

Area	Coverage
Bus & reset	Address decoding of all device windows; reset-vector fetch; ROM-scripted bus cycles
VIA 6522	Port masking; IER/IFR; Timer-1 IRQ assertion and clearing
UART 6551	Status, TX, RX, overrun, programmed reset, RX IRQ
T65 CPU	Boot from real ROM; UART/VIA access; IRQ handler; kernel smoke test
D64 / FAT32	Real-geometry images read lazily; start LBAs verified
SID & FPU	SID audio/filter bounded and non-silent; 8.24 multiply checked against a reference

Table 2: *Incremental GHDL testbench coverage, from address decoding up to the FAT32 reader.*

13 Portability: One Core, Two Boards

The board-independent RTL core runs on two FPGA targets with only thin board-specific glue, summarised in Table 3.

	Tang Primer 20K *active bring-up*	**PIX16 board** *reference*
FPGA	Gowin GW2A-18	Xilinx Spartan-6 xc6slx16
Video	HDMI/DVI via TMDS	VGA text + boot status
Main RAM	BSRAM (DDR3 optional)	SDRAM with self-test
Boot	On-board microSD (SPI)	SD boot of 16 KB shadow ROM
Sound	Native SID via PT8211 DAC	—
Input	PS/2 over PMOD, CH340 UART	UART monitor
Toolchain	Gowin project	ISE project (versioned)

Table 3: *The two FPGA platforms. The Tang Primer 20K is the active hardware bring-up target; the Spartan-6 PIX16 serves as a reproducible reference with a live ROM-upload monitor.*

14 The Reference Emulator

The FPGA design grew out of a companion emulator, a C99 program that came first. It runs the MOS 6502 with all 151 official opcodes, models the same memory-mapped VIA, UART, VIC and sound devices, draws through SDL2, and carries an interactive monitor: registers, memory dumps, disassembly, single-stepping, breakpoints. It ships with ready-to-run setups for MS BASIC and a standalone chess ROM, under the MIT licence. The FPGA build includes the emulator's ROM and kernel sources as a submodule, which is the mechanism that keeps both sides on the same firmware.

15 Results and Current Status

The machine runs on real hardware. On the Tang Primer 20K it boots EhBASIC to an HDMI display, takes input from a PS/2 keyboard, loads programs from SD through GoDrive, plays original `.sid` music through the DAC, and draws 256-colour FPU-accelerated Mandelbrot sets, all from the same ROM images the emulator uses. Along the way the individual milestones have each been checked: the HDMI boot screen, the CH340 serial port, the UART monitor that can hold the CPU and hot-upload a ROM, the PS/2 keyboard injection, and the diagnostic that reports correctly when no SD card is present. Figure 7 shows that self-test on the HDMI output.

16 Future Work

The roadmap continues to deepen the peripheral set rather than chase a specific historical machine. The VIC-II compatibility layer already provides border/background colour and live raster read-back; the natural next step is a true **raster-compare interrupt** (with writable RAM under the kernel vectors), which would let raster-timed players run from their own interrupt instead of the polled work-around of Section 7. Further directions include VIC sprites and a blitter, a fuller CIA implementation (Timer B, TOD), DDR3 as the default main-memory backend, and continued hardening of the clock-domain and constraint work across both boards. A known open item is a T65 indirect-addressing integration case (`STA (zp),Y` into video RAM), currently quarantined as an experimental testbench outside the default suite.

Figure 7: *The power-on self-test on the HDMI output, reporting RAM, video RAM, VIA and UART results before control is handed to the application ROM.*

17 Conclusion

Treating the memory map as a contract and building to it twice keeps the emulator and the FPGA computer in step without forcing either to give up what it is good at. What comes out is a 6502 home computer that is modern where modernity earns its place, with HDMI, SD storage and a hardware multiplier, yet stays close enough to the old machines that decades-old software, player routines and tooling still feel at home on it. I am glad to hear feedback and ideas from anyone working with the 6502.

Project Resources

- Project page: intracom.at/projects/6502_fpga_computer
- FPGA (RTL/sim/boards): github.com/rudolfstepan/6502-sbc-fpga
- Software emulator: github.com/rudolfstepan/6502-sbc-emulator
- Author: Rudolf Stepan — ORCID 0009-0004-2842-2579 • stepan.science

References

[1] R. Stepan, “6502 SBC FPGA — Synthesizable VHDL 6502 computer,” project page and source repository. github.com/rudolfstepan/6502-sbc-fpga

[2] R. Stepan, “6502 SBC Emulator — MOS 6502 SBC emulator in C99,” source repository. github.com/rudolfstepan/6502-sbc-emulator

[3] D. Wallner, “T65 — 6502-compatible CPU core in VHDL,” OpenCores.

[4] L. Davison, “EhBASIC — Enhanced 6502 BASIC.”

[5] K. Dormann, “6502/65C02 functional tests.”

[6] MOS Technology, *MCS6500 Microcomputer Family Programming Manual* (6502 CPU).

[7] MOS Technology, *6581 SID (Sound Interface Device) datasheet*.

[8] MOS Technology, *6522 Versatile Interface Adapter (VIA) datasheet*.

[9] MOS Technology, *6551 Asynchronous Communication Interface Adapter (ACIA) datasheet*.

[10] Sipeed, *Tang Primer 20K (Gowin GW2A-18) documentation*.

[11] T. Gleixner et al., “GHDL — open-source VHDL simulator.”